folio sample document

This is a multi-page sample used to exercise folio's rendering and page navigation. The same content is provided in three parallel formats — Markdown, LaTeX, and PDF — so you can compare how each input path looks. Page with `n` / `p`, jump with `:n`, and zoom with `+` / `-`.

1 Introduction

folio renders document pages as images directly in your terminal using the sixel or kitty graphics protocols. It opens PDFs natively, converts Markdown to PDF on the fly, and—when you opt in via the `latex_engine` setting—compiles LaTeX with an installed TeX engine. Whatever the source, the same viewer provides zoom, pan, dark mode, configurable sharpness, bookmarks, and resume.

This document was typeset by a real TeX engine and then rasterized by folio. The text is intentionally long enough to span more than two pages so that multi-page behaviour is easy to try out.

2 Text formatting

Paragraphs may contain **bold**, *italic*, ***bold italic***, and inline `code` such as `let x = 42;`. Long paragraphs wrap to the page width and are justified by the TeX engine, breaking cleanly and hyphenating where appropriate rather than running past the margin.

Source line breaks are collapsed into the same paragraph, so you can keep your LaTeX lines short without affecting the rendered output; a blank line starts a new paragraph.

3 Lists

Unordered lists, including nested items:

- The quick brown fox jumps over the lazy dog.
- A second item with a little more text so it wraps to demonstrate hanging indentation under the bullet.
  - A nested item.
  - Another nested item.
- A third top-level item.

Ordered lists:

1. Preheat the oven.
2. Mix the ingredients thoroughly.
3. Bake until golden, then let cool before serving.

4 Code

Verbatim text renders in a monospace face:

```
fn main() {
    let pages = vec!["one", "two", "three"];
    for (i, name) in pages.iter().enumerate() {
        println!("page {}: {}", i + 1, name);
    }
}
```

5 A longer discussion

Terminal document viewers occupy a useful niche: they keep you in the keyboard-driven, text-first environment where so much development happens, while still showing real, faithfully rendered pages. Rather than dumping a lossy text approximation of a document, folio rasterizes the genuine page and paints it with a graphics protocol, so figures, fonts, and layout survive intact.

Because only the page under view is rendered on demand, memory and CPU stay modest even for large documents, and an LRU cache plus predictive prefetch keep navigation feeling immediate. The same machinery that makes PDFs responsive applies to converted Markdown and LaTeX, since every input is normalised to PDF before it reaches the renderer.

This uniformity is the point of the modular input layer. Adding a new input format is a matter of teaching folio how to turn that format into PDF bytes; the viewer, the cache, the themes, and the bookmark store do not need to know where those bytes came from. Markdown was the first such format, LaTeX the second, and the same seam is ready for whatever comes next.

6 Mathematics

Unlike the Markdown path, the LaTeX path typesets mathematics properly. Inline math such as $E = mc^2$ and $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ sits naturally within the prose, with Greek letters like α, β, and π rendered as real glyphs.

A displayed equation:

$$\int_0^1 x^2 \, dx = \frac{1}{3}, \qquad e^{i\pi} + 1 = 0. \tag{1}$$

7 Conclusion

If you can see this section, pagination is working and you have paged past the first page. Try toggling dark mode with `i`, adjusting sharpness with `[` and `]`, dropping a bookmark with `m`, and reopening the file to confirm it resumes where you left off.